

Exercício Prático: API de Empréstimos da Biblioteca Lions

Turma: LionsDev

Tópicos: APIs REST com Express.js, Integração com MongoDB (Mongoose), Múltiplos Schemas, Validações com Mongoose, Operações CRUD assíncronas (async/await), Query Params (filtros), Validação cruzando coleções, Controle de estoque e Status Codes HTTP.

1. Contexto

A **Biblioteca Lions** controla seus empréstimos em uma planilha simples. O problema é que os registros ficam desorganizados: a equipe não sabe rapidamente quais materiais estão emprestados, devolvidos ou atrasados, e às vezes empresta um material que **não tem mais exemplares disponíveis**.

Você deverá criar uma API REST conectada ao MongoDB com **dois recursos**: o **Material** (o acervo) e o **Empréstimo**. Antes de emprestar, a API precisa checar se o material existe e se ainda há exemplares em estoque. A estrutura deve seguir o modelo usado em aula: `server.js`, `db.js` e `models/`, com as rotas escritas diretamente no `server.js`.

Nível: este exercício é um degrau acima dos demais (Cantina, Petshop, Academia). São **dois models** e uma regra que cruza coleções, mas ainda **sem** relacionamentos com **ObjectId** / **populate** (isso fica como bônus).

2. Configuração Inicial e Estrutura de Pastas

Crie uma API Node.js com Express e Mongoose, com a seguinte estrutura:

```
BibliotecaLions/
|-- src/
|   |-- models/
|   |   |-- material.js
|   |   |-- emprestimo.js
|   |-- db.js
|   |-- server.js
|-- .env
|-- package.json
```

Requisitos estruturais:

- Crie um arquivo `.env` contendo as variáveis `MONGO_URI` e `PORT` (porta `3000`).
- Crie o arquivo `src/db.js` para gerenciar a conexão com o MongoDB.

- Crie o arquivo `src/server.js` como ponto de entrada da aplicação.
- Crie a pasta `src/models` com os arquivos `material.js` e `emprestimo.js`.
- Todas as rotas devem ficar no `src/server.js`, como fizemos em sala.

2.1 Modelos de Dados (Schemas)

Modelo: Material (`models/material.js`)

- `titulo`: Tipo `String`, obrigatório.
- `tipo`: Tipo `String`, obrigatório (deve aceitar apenas: `Livro`, `Revista` ou `Apostila`).
- `autor`: Tipo `String`, obrigatório.
- `estoque`: Tipo `Number`, obrigatório (quantidade de exemplares disponíveis para empréstimo).

Modelo: Empréstimo (`models/emprestimo.js`)

- `materialId`: Tipo `String`, obrigatório. Esse campo deve guardar o `_id` do material emprestado.
 - `nomeAluno`: Tipo `String`, obrigatório.
 - `turma`: Tipo `String`, obrigatório.
 - `dataEmprestimo`: Tipo `String`, obrigatório (ex: `"2026-06-15"`).
 - `diasEmprestimo`: Tipo `Number`, obrigatório.
 - `multaPrevista`: Tipo `Number` (será calculado automaticamente pela API).
 - `status`: Tipo `String`, com valor padrão de `"Emprestado"` (deve aceitar apenas: `Emprestado`, `Devolvido` ou `Atrasado`).
-

3. Requisitos Obrigatórios e Regras de Negócio

Você deverá implementar endpoints para gerenciar Materiais (`/materiais`) e Empréstimos (`/emprestimos`).

3.1 CRUD de Materiais

- **Cadastrar Material (`POST /materiais`)**: Cria um novo material no acervo, com a quantidade inicial de `estoque`. Retorne status `201`.
- **Listar Materiais (`GET /materiais`)**: Retorna todos os materiais cadastrados.
- **Listar Materiais Disponíveis (`GET /materiais/disponiveis`)**: Retorna apenas os materiais com `estoque` maior que zero.

3.2 Empréstimos

- **Cadastrar Empréstimo (`POST /emprestimos`)**:

O corpo da requisição enviará os dados do empréstimo (exceto `multaPrevista` e `status`).

```
{
  "materialId": "65b...d1a",
  "nomeAluno": "Mariana Souza",
  "turma": "LionsDev",
  "dataEmprestimo": "2026-06-15",
  "diasEmprestimo": 10
}
```

Regras de Negócio (Lógica em JavaScript):

1. **Validação de Existência:** Verifique se o material informado existe no banco. Se não existir, retorne status **404** com erro.
 2. **Validação de Estoque:** Verifique se o material ainda tem **estoque** disponível (maior que zero). Se não tiver, retorne status **400** com uma mensagem (ex: **"Material sem exemplares disponíveis"**).
 3. **Multa Prevista:** A biblioteca permite até 7 dias de empréstimo sem multa. Se **diasEmprestimo** for maior que 7, cobre R\$ 2 por dia extra.
 4. **Baixa no Estoque:** Se as regras passarem, diminua em 1 o **estoque** do material e salve o empréstimo com o **multaPrevista** calculado. Retorne o documento com status **201**.
- **Listar Empréstimos (GET /emprestimos):** Retorna todos os empréstimos cadastrados.
 - **Buscar Empréstimos por Aluno (GET /emprestimos/busca):**
 - Aceita um filtro opcional via Query Param chamado **aluno**.
 - Exemplo de URL: **http://localhost:3000/emprestimos/busca?aluno=mari**
 - Retorna todos os empréstimos em que o nome do aluno contenha o texto pesquisado.
 - **Registrar Devolução / Alterar Status (PATCH /emprestimos/:id/status):**
 - O corpo envia apenas o novo **status** (ex: **{ "status": "Devolvido" }**).
 - Se o ID não for encontrado, responda com status **404**.
 - **Regra de Estoque:** quando o status mudar para **"Devolvido"**, devolva 1 ao **estoque** do material correspondente.
 - Retorne o documento atualizado com status **200**.
 - **Remover Empréstimo (DELETE /emprestimos/:id):**
 - Remova o empréstimo do banco pelo ID usando **findByIdAndDelete**.
 - Se não encontrar o registro, retorne status **404**.
 - Se remover com sucesso, retorne status **200** com uma mensagem de confirmação.
 - **Relatório (GET /emprestimos/relatorio):**

- Calcule em JavaScript e retorne: total de empréstimos, soma das multas previstas e a contagem de empréstimos por status (quantos **Emprestado**, **Devolvido** e **Atrasado**).

4. Desafios Bônus

Depois que a versão obrigatória estiver funcionando, tente adicionar uma ou mais melhorias:

- Bloquear um novo empréstimo se o mesmo aluno já tiver um empréstimo com status **"Emprestado"** do mesmo material.
- Criar **GET /materiais/:id/emprestimos** que lista o histórico de empréstimos de um material.
- **Avançado:** usar **mongoose.Schema.Types.ObjectId**, **ref** e **.populate()** no campo **materialId** para trazer os dados completos do material ao listar os empréstimos. Exemplo mínimo:

```
// models/emprestimo.js
materialId: { type: mongoose.Schema.Types.ObjectId, ref: "Material", required: true },

// server.js
const emprestimos = await Emprestimo.find().populate("materialId");
```

5. Testes Esperados

Realize testes na sua API seguindo este fluxo de validação:

1. Cadastre um material **Livro** com **estoque** igual a 1.
2. Liste os materiais disponíveis (**GET /materiais/disponiveis**) e confira que ele aparece.
3. Crie um empréstimo de 5 dias para esse material e verifique se a **multaPrevista** ficou R\$ 0.
4. Confira que o **estoque** do material caiu para 0.
5. Tente criar outro empréstimo do mesmo material (deve falhar com erro **400** por falta de estoque).
6. Crie um material com mais estoque e faça um empréstimo de 10 dias; verifique se a **multaPrevista** ficou R\$ 6.
7. Faça uma busca pelo nome de um aluno.
8. Mude o status de um empréstimo para **"Devolvido"** e confira que o **estoque** do material voltou a subir.
9. Consulte o relatório (**GET /emprestimos/relatorio**) e confira os totais.
10. Delete um dos empréstimos informando o ID.

6. Dicas para a Implementação

- No **src/server.js**, importe os dois Models:

```
import Material from "../models/material.js";  
import Emprestimo from "../models/emprestimo.js";
```

- Na rota de criar empréstimo, busque o material antes de validar o estoque:

```
const material = await Material.findById(materialId);
```

- Para dar baixa/retorno no estoque, altere o campo e salve:

```
material.estoque = material.estoque - 1;  
await material.save();
```

- Trate `dataEmprestimo` apenas como texto. Não precisa fazer cálculo com data neste exercício.
 - Para calcular a multa, use uma estrutura `if` com o campo `diasEmprestimo`.
 - No `findByIdAndUpdate`, passe `{ new: true, runValidators: true }`.
 - Lembre-se de usar `app.use(express.json())` no `src/server.js` para conseguir ler o corpo das requisições.
-